Laboratorium 07 – Listy i układy

ListView, GridView i proste interakcje

Na tym labie poznasz wygodne sposoby wyświetlania wielu elementów. Zobaczysz listy pionowe i poziome, siatkę kafelków oraz prostą reakcję na kliknięcie.

---

1. Starter projektu

Użyj szablonu z AppBarem. W `home` wstawimy różne przykłady.

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: const Lab07Home(),
      theme: ThemeData(useMaterial3: true, colorSchemeSeed:
Colors.deepPurple),
    );
  }
}

class Lab07Home extends StatelessWidget {
  const Lab07Home({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Lab 07 – Listy i układy')),
      body: const Center(child: Text('Podmień body przykładem z sekcji
poniżej')),
    );
```

```
  }
}
// tutaj wklej class z przykładów ponizej np. class VerticalListExample
extends StatelessWidget { ...
```

---

2. ListView.builder pionowo

Do długich list używamy `ListView.builder`. Tworzy elementy dopiero wtedy, gdy są potrzebne.

```dart
class VerticalListExample extends StatelessWidget {
  const VerticalListExample({super.key});

  @override
  Widget build(BuildContext context) {
    final items = List<String>.generate(20, (i) => 'Element #$i');

    return ListView.builder(
      padding: const EdgeInsets.all(12),
      itemCount: items.length,
      itemBuilder: (context, index) {
        return Card(
          child: ListTile(
            title: Text(items[index]),
            subtitle: const Text('Podtytuł'),
            trailing: const Icon(Icons.chevron_right),
            onTap: () {
              ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(content: Text('Kliknięto ${items[index]}')),
              );
            },
          ),
        );
      },
    );
  }
}
```

👉 **Wyjaśnienie**:

- `ListView.builder` buduje w locie tylko widoczne elementy
- `itemBuilder` przyjmuje kontekst i indeks
- `SnackBar` pokazuje krótką informację po tapnięciu

Zamień `body` w `Lab07Home` na const `VerticalListExample`().

---

3. ListView poziomo

Przydatne do karuzel. Używamy `scrollDirection`: `Axis.horizontal`.

```
class HorizontalListExample extends StatelessWidget {
  const HorizontalListExample({super.key});

  @override
  Widget build(BuildContext context) {
    final colors = [
      Colors.red, Colors.green, Colors.blue, Colors.orange, Colors.purple,
    ];

    return SizedBox(
      height: 120,
      child: ListView.separated(
        scrollDirection: Axis.horizontal,
        padding: const EdgeInsets.all(12),
        itemCount: colors.length,
        separatorBuilder: (_, __) => const SizedBox(width: 12),
        itemBuilder: (context, i) {
          return Container(
            width: 160,
            decoration: BoxDecoration(
              color: colors[i],
              borderRadius: BorderRadius.circular(16),
            ),
            alignment: Alignment.center,
            child: Text(
              'Karta ${i + 1}',
              style: const TextStyle(color: Colors.white, fontSize: 18),
            ),
          );
        },
      ),
```

```
    );
  }
}
```

👉 **Wskazówka:**

- `ListView.separated` dodaje odstępy między elementami bez owijania w dodatkowe widgety

---

4. GridView siatka kafelków

Dobre do galerii. Poniżej przykładowa siatka kolorów.

```
class GridExample extends StatelessWidget {
  const GridExample({super.key});

  @override
  Widget build(BuildContext context) {
    final tiles = List.generate(
      12,
      (i) => Colors.primaries[i % Colors.primaries.length],
    );

    return GridView.builder(
      padding: const EdgeInsets.all(12),
      gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
        crossAxisCount: 3,      // liczba kolumn
        crossAxisSpacing: 8,
        mainAxisSpacing: 8,
      ),
      itemCount: tiles.length,
      itemBuilder: (context, i) {
        return InkWell(
          onTap: () {
            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(content: Text('Kafelek #$i')),
            );
          },
          child: Container(
            decoration: BoxDecoration(
              color: tiles[i],
              borderRadius: BorderRadius.circular(12),
            ),
```

```
          ),
        );
      },
    );
  }
}
```

👉 **Wskazówka:**

- `SliverGridDelegateWithFixedCrossAxisCount` ustala stałą liczbę kolumn
- `InkWell` daje efekt dotyku

---

5. Łączenie układów

Często chcesz mieć listę z nagłówkiem i poziomą karuzelą. Najprościej można to osiągnąć z `CustomScrollView` lub luźniej na start z pojedynczym `ListView` i kilkoma `children`.

```
class MixedLayoutsExample extends StatelessWidget {
  const MixedLayoutsExample({super.key});

  @override
  Widget build(BuildContext context) {
    return ListView(
      padding: const EdgeInsets.all(12),
      children: const [
        Text('Polecane', style: TextStyle(fontSize: 22, fontWeight:
FontWeight.bold)),
        SizedBox(height: 8),
        HorizontalListExample(),
        SizedBox(height: 16),
        Text('Wszystkie', style: TextStyle(fontSize: 22, fontWeight:
FontWeight.bold)),
        SizedBox(height: 8),
        SizedBox(height: 400, child: VerticalListExample()),
      ],
    );
  }
}
```

Zadania praktyczne 💻

Wybierz `body` i wstaw kod startowy z przykładu.

Zadanie 1: Karty z produktami

📍 Start od `VerticalListExample`.

- Zamiast Element przygotuj listę map z polami `title` i `price`
- W `ListTile` pokaż cenę w `trailing` oraz ikonę koszyka w `leading`
- Po kliknięciu w element wyświetl `SnackBar` z ceną i tytułem

```
ListTile(
  leading: ...
  title: ...
  subtitle: ...,
  trailing: ...,
  onTap: () {...,
        ),
      ),
    );
  },
)
```

Zadanie 2: Karuzela promocyjna

📍 Start od `HorizontalListExample`.

- Zamiast kolorów użyj `Image.network` z trzema różnymi obrazami
- Dodaj zaokrąglone rogi i cień przez `ClipRRect` i `BoxShadow`
- Po tapnięciu pokaż `SnackBar` z numerem slajdu

Przykładowe obrazki do zadania 2 i 3:

```
final List<String> obrazki = [
  'https://picsum.photos/id/1015/400/300',
  'https://picsum.photos/id/1025/400/300',
  'https://picsum.photos/id/1035/400/300',
  'https://picsum.photos/id/1045/400/300',
  'https://picsum.photos/id/1055/400/300',
  'https://picsum.photos/id/1065/400/300',
];
```

Zadanie 3: Mini galeria w siatce

📍 Start od `GridExample`.

- Zamiast kolorów użyj listy adresów zdjęć
- Dodaj nakładkę w prawym dolnym rogu z ikoną serca
- Po tapnięciu serca zmień stan wyłącznie dla danego kafelka i zaktualizuj ikonę

  Podpowiedź
  - owiń kafelek w `StatefulBuilder` albo przenieś kafelek do małego `StatefulWidget`

Zadanie 4: Sekcja mieszana

📍 Start od `MixedLayoutsExample`.

- U góry nagłówek z liczbą elementów w liście poniżej
- Po przewinięciu listy zmień tytuł AppBar na Przewinięto

  ℹ️ Podpowiedź
  - użyj NotificationListener<`ScrollNotification`> i `setState` w rodzicu

Przydatne linki 🔗

- Layout i budowanie widoków - https://docs.flutter.dev/ui/layout
- ListView - https://api.flutter.dev/flutter/widgets/ListView-class.html
- GridView - https://api.flutter.dev/flutter/widgets/GridView-class.html
- SnackBar - https://api.flutter.dev/flutter/material/SnackBar-class.html